Wednesday 15th of December 2021
9:00 am — 11:30am
Duration: 2 hours
Additional time: 30 minutes
Timed exam — fixed start time
—
!!!!!!!!!!!!!!!! SOLUTIONS !!!!!!!!!!!!!!!

DEGREE OF MSc

INTRODUCTION TO DATA SCIENCE AND SYSTEMS (M)
COMPSCI5089

Answer all 3 questions

This examination paper is an open book, online assessment and is worth a total of 60 marks.

CONTINUED OVERLEAF

1. Computational linear algebra and optimisation

(a) Given a collection of N documents $\mathscr{D} = \{D_1, \ldots, D_N\}$, your task is to implement a functionality that provides a list of suggested 'more like this' documents. With this problem context, answer the following questions.

(i) Explain how would you represent each document $D \in \mathscr{D}$ as a (real-valued) vector $\mathbf{d}$. What is the dimension of each vector? [2]

> **Solution:** First, let $\mathscr{V}$ denote the vocabulary set of the collection, i.e., the set of all unique terms in $\mathscr{D}$. Each document D can then be converted into a vector $\mathbf{d} \in \mathbb{R}^{|\mathscr{V}|}$. [1] Each component of a vector $\mathbf{d}$, denoted as $\mathbf{d}_i$, refers to the 'weights' of a particular term. Among various choices, one can use a simple Boolean presence/absence as the weights, in which case the vector $\mathbf{d}$ of a document D takes up the form
>
> $$\mathbf{d}_i = \mathbb{I}(t_i \in D),\ \forall t_i \in \mathscr{V},$$
>
> where $\mathbb{I}(X) = 1$ if a property X is true or $\mathbb{I}(X) = 0$ otherwise. [1] One may also consider the term frequencies instead of a simple term presence to construct the document vectors.
>
> **Marking**: Check if they correctly define the document vectors as **vocabulary-dimensional**. It's okay to use any term weights, like Boolean or raw frequency, or relative frequency etc.

(ii) What does the L_0 norm of a document vector indicate (in plain English) as per your definition of the document vectors in the previous question? [1]

> **Solution:** The L_0 norm in this case would indicate the number of terms present in a document, or in other words, the document length. [1]
>
> **Marking**: Look for the presence of *number of terms within a document* or *document length*.

(iii) How would you define the L_p distance between two document vectors $\mathbf{d}$ and $\mathbf{d}'$? [2]

> **Solution:** The L_p norm of a document vector $\mathbf{d}$ is defined as
>
> $$L_p(\mathbf{d}) = \sum_{i=1}^{|\mathscr{V}|} |\mathbf{d}_i|^p$$
>
> The L_p distance between $\mathbf{d}$ and $\mathbf{d}'$ is defined as the L_p norm of their difference vector, $\mathbf{d} - \mathbf{d}'$, i.e.,
>
> $$||\mathbf{d} - \mathbf{d}'||_p = (\sum_{i=1}^{|\mathscr{V}|} |\mathbf{d}_i - \mathbf{d}'_i|^p)^{\frac{1}{p}}.$$
>
> [2]

Marking: Deduct 1 marks if the answer uses variables other than $\mathbf{d}$ and $\mathbf{d}'$ for defining the L_p distance. It should also use the same symbol for dimensionality (as asked in the first question).

(iv) What distance or similarity measure would you use for finding the set of 'more like this' documents for a current (given) document vector $\mathbf{d}$, and why. [2]

Solution: Since documents can considerably vary in length, L_2 distance (or any other distance measure) is not effective. The solution is to use a length independent (normalized) measure. [1] Cosine similarity is one such similarity function because it is defined as

$$s(\mathbf{d},\mathbf{d}') = \frac{\mathbf{d}\cdot\mathbf{d}'}{|\mathbf{d}||\mathbf{d}'|}.$$

[1] Another possibility is to use L_2 (or L_p for any $p > 0$) if the vectors are length normalized.

Marking: Check if they mention *length differences*. If the answer is a distance function, the normalization solution must be mentioned as well for securing full marks.

(b) The probability distribution function of an n dimensional Gaussian is given by

$$f(\mathbf{x}) = (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu),$$

where $\mu \in \mathbb{R}^n$ is the mean vector and $\Sigma \in \mathbb{R}^{n \times n}$ is a square and invertible matrix, called the *covariance* matrix. Consider the particular case of $n = 2$. Answer the following questions.

(i) Plot the contours of the following Gaussians. For each contour plot, show the conditional distributions along the two axes.

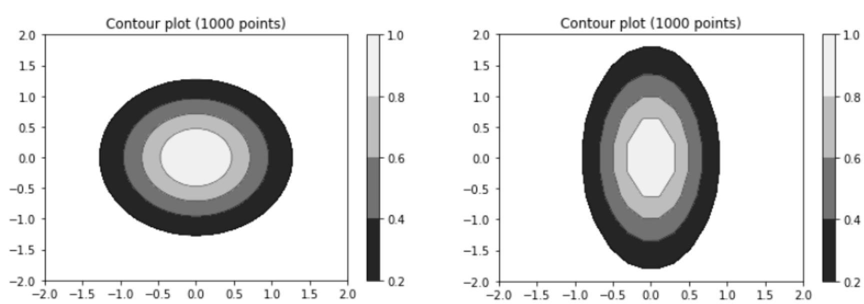
$$\mu = (0,0)$$
$$\Sigma = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\mu = (0,0)$$
$$\Sigma = \begin{pmatrix} 0.5 & 0 \\ 0 & 2 \end{pmatrix}$$

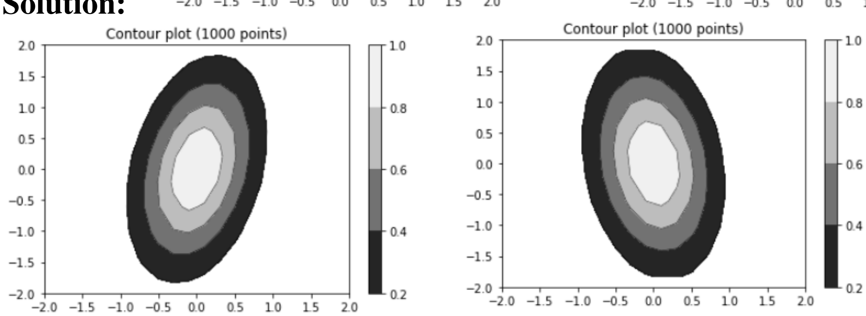
$$\mu = (0,0)$$
$$\Sigma = \begin{pmatrix} 0.5 & 0.1 \\ 0.5 & 2 \end{pmatrix}$$

$$\mu = (0,0)$$
$$\Sigma = \begin{pmatrix} 0.5 & 0.1 \\ -0.5 & 2 \end{pmatrix}$$

[2]



Solution:



Marking: 0.5 marks each. The first plot should actually be a set of concentric circles (for some reasons, pyplot doesn't show this as a circle to me). For the last two plots, check the tilt axis (don't deduct marks if the absolute value of the slopes don't match with the answers shown here).

(ii) Which one/ones of the above 4 Gaussian distributions can be reduced to a a single dimensional Gaussian with PCA on the covariance matrix without too much loss of information. Note that you do not need to explicitly compute the Eigenvalues. You should rather derive your answer from a visual interpretation of the contour plots. Clearly explain your answer. [2]

Solution: The third and the fourth Gaussians, because they have their axes tilted towards a direction that's not parallel to either of the basis vectors. In both these cases projecting the data along the y-axis would capture maximum variance in the data. [2]

(c) With respect to linear regression, answer the following questions.

(i) Derive the expression for stochastic gradient descent for linear regression with the squared loss function. Clearly introduce your notations for the input/output instances, and the parameter vector. [4]

Solution: Let $X = \{\mathbf{x}_1, \ldots \mathbf{x}_M\}$, be a set of given points (input data instances), where $\mathbf{x}_i \in \mathbb{R}^n$ and a set $Y = \{y_1, \ldots, y_M\}$ of associated values, The objective in linear regression is to fit a curve that best fits the curve (minimizes the sum of the distances drawn from each point to the closest point in the curve). Let the curve itself be parameterized by θ, a general form of which is

$$h_\theta(\mathbf{x}) = \theta_0 + \theta_1.x_1 + \ldots \theta_1.x_n = \theta_0 + \theta^T \cdot \mathbf{x}.$$

By augmenting a '1' as the first component in $\mathbf{x}$ (thus making it an $n+1$ dimensional vector), we can simply write

$$h_\theta(\mathbf{x}) = \theta^T \cdot \mathbf{x}.$$

[1]
The squared error for an instance $\mathbf{x}$ (with ground-truth value y) is given by:

$$L(\mathbf{x}, y; \theta) = \frac{1}{2}(\theta^T \cdot \mathbf{x} - y_i)^2.$$

[1]
In stochastic gradient descent, we update each component of the parameter vector by taking a step towards the direction that results in minimizing the loss function, i.e,

$$\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} + \alpha \frac{\partial}{\partial \theta_j} L(\theta),$$

where $\theta_j^{(t)}$ denotes the value of the j^{th} component of the parameter vector at iteration t. [1]
The partial derivative of the loss function is given by:

$$\begin{aligned}\frac{\partial}{\partial \theta_j} L(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2}(\theta^T \mathbf{x} - y)^2 \\ &= 2.\frac{1}{2}(\theta^T \mathbf{x} - y) \frac{\partial}{\partial \theta_j}(\theta^T \mathbf{x} - y) \\ &= (\theta^T \mathbf{x} - y) x_j.\end{aligned}$$

[1]
Marking: Check if they correctly derive the value of the partial derivative of the loss function. It's okay to be a bit lenient on the variable definitions.

(ii) Explain how linear regression can be extended to polynomial (higher order) regression? What is the problem of using high degree polynomials for regression? How can that problem be alleviated? [3]

> **Solution:** Yes, the easiest way is to include the higher order feature values as additional dimensions in $\mathbf{x}$, e.g., for a two dimensional vector (x_1, x_2), we would then augment this vector as (x_1, x_2, x_1^2, x_2^2), and apply linear regression. [1] This means that there will be parameter vectors present for the higher order features as well, like θ_3 for x_1^2 in the example.
> The problem of using higher order polynomials is that they **tend to overfit** a set of given observations and hence is likely **not to generalize well** for unseen observations. [1]
> This can be alleviated by adding a **penalization term** in the loss function to achieve **sparse** parameter vector solutions. [1]

(iii) A common practice in stochastic gradient descent is to use a variable learning rate α for the parameter updates

$$\theta_j^{(t+1)} \leftarrow \theta_j^{(t)} + \alpha^{(t)} \frac{\partial L}{\partial \theta_j},$$

where $\theta_j^{(t)}$ denotes the j^{th} component of the parameter vector θ at iteration t, and $\alpha^{(t)}$ denotes the value of the learning rate at iteration t. Which of the following alternatives of the learning rate update would you prefer (α is a constant) and why?

$$a)\, \alpha^{(t)} = \frac{\alpha}{t}, \quad b)\, \alpha^{(t)} = \alpha + t.$$

[2]

> **Solution:** The learning rate should be a **monotonically decreasing** with an increase in the iterations. [1] Option (b) leads to increasing the learning rate with iterations. Hence, option (a) is the correct choice. [1]

2. Probabilities & Bayes rule

Consider a card game where you have 4 suits (heart, diamonds, clubs and spades) and in each suit the cards 7, 8, 9, 10, Jack (J), Queen (Q), King (K) and Ace (A). In this question we will use the following commonly used terms:

- the *the pack*: is the set of all cards that have not been drawn yet.
- to *draw*: is to pick a card at random amongst the pack of remaining cards, removing it from the pack.
- the *hand*: is the set of cards a player has drawn from the pack.
- a *payout*: is the amount of points you get for a given hand.
- to *fold*: is to stop playing and put back your cards in the pack, forfeiting any payout for this game.

(a) Assuming that you draw a single card at random from the pack, give the probabilities for the following events

(i) Drawing an Ace?

> **Solution:** $p(A) = \frac{4}{32} = \frac{1}{8}$ [1]

(ii) Drawing a red card?

> **Solution:** $p(A) = \frac{16}{32} = \frac{1}{2}$ [1]

(iii) Drawing a diamonds?

> **Solution:** $p(A) = \frac{8}{32} = \frac{1}{8}$ [1]

(iv) Drawing a royalty figure (Jack, Queen or King)?

> **Solution:** $p(A) = \frac{3\times 4}{32} = \frac{3}{8}$ [1]

(v) Drawing the Ace of spades?

> **Solution:** $p(A) = \frac{1}{32}$ [1]

[5]

(b) Now assume that you have already drawn the three cards: 10,J,Q. When drawing two more cards from the pack, what is the probability to obtain:

(i) A pair of two cards with the same value (eg, two Jacks).

> **Solution:**
> $$p(\text{pair}) = \frac{3\times 3}{32-3} + \left(1 - \frac{3\times 3}{32-3}\right)\frac{4\times 3}{32-4} \simeq 0.61$$

or

$$p(\text{pair}) = 1 - p(\text{not pair}) = 1 - \frac{5\times 4}{32-3}\frac{4\times 4}{32-4} \simeq 1 - 0.39 \simeq 0.61$$

[1]

(ii) Two pairs (eg, two Jacks and two Queens).

Solution: $p(\text{twopairs}) = \frac{3\times 3}{32-3} \times \frac{2\times 3}{32-4} \simeq 0.07$ [1]

(iii) Three of a kind (eg, three Jacks).

Solution: $p(\text{three}) = \frac{3\times 3}{32-3} \times \frac{2}{32-4} \simeq 0.02$ [1]

(iv) A sequence of 5 cards (eg, 10, J, Q, K, A). Note that the cards can be of any suit, but there cannot be a break in the sequence.

Solution: $p(\text{sequence}) = p(< 8, 9, 10, J, Q >) + p(< 9, 10, J, Q, K >) + p(< 10, J, Q, K, A >) = \frac{4*4}{29} \times \frac{4*4}{28} + \frac{4*4}{29} \times \frac{(4+4)*4}{28} + \frac{4*4}{29} \times \frac{(4+4)*4}{28} + \frac{4*4}{29} \times \frac{4*4}{28} = 0.12$ [1]

[4]

(c) Now let us assume the following payout table for each hand of 5 cards:

hand	payout
sequence of 5 cards	50
three of a kind	30
two pairs	20
one pair	10
anything else	0

As before, you have the cards 10, J, Q in hand.

(i) If you draw two more cards randomly from the deck, what is the expected value of the payout for this hand? [3]

Solution: $\mathbb{E}[payout] = 10p(\text{pair}) + 20p(\text{twopairs}) + 30p(\text{three}) + 50p(\text{sequence}) \simeq 14/1$ [2] for the formulation [1] for the result.

(ii) Assuming that you need to pay 5 every time you draw a card (hence you would need to pay 10 to draw two cards), should you fold your hand or draw cards? [2]

Solution:

$$\mathbb{E}[\text{payou}] = 10p(\text{pair}) + 20p(\text{twopairs}) + 30p(\text{three}) + 40p(\text{sequence}) \simeq 13$$

Given the expected return, you should draw two more cards.

(iii) Should you fold after drawing the first card (and having paid 5), if the card is: (i) the 7 of heart, (ii) the 8 of spades or (iii) the Queen of diamonds? [6]

Solution:
After a 7, the only winning hands would require to draw a pair $\frac{4*3}{28}$, so the expectation becomes:

$$\mathbb{E}[\text{payou}] = 10\frac{4\times 3}{28} \simeq 4$$

[1], so the player is better off folding now to limit losses. [1]
After an 8, the winning hands would require to draw a pair $\frac{4\times 3}{28}$ or an 8 $\frac{4}{28}$, so the expectation becomes:

$$\mathbb{E}[\text{payout}] = 10\frac{4\times 3}{28} + 40\frac{4}{28} \simeq 10$$

[1], so you should draw another card hoping to break even. [1]
After a queen, you are assured a pair and the possible outcomes become: three of a kind if you draw another queen $\frac{2}{28}$, two pairs if you draw 10 or J $\frac{3*2}{28}$, and a pair otherwise. The expectation becomes: $\mathbb{E}[\text{payout}] = 30\frac{2}{28} + 20\frac{3\times 2}{28} + 10\frac{5\times}{28} \simeq 13$ [1], so it is worth drawing another card. [1]

3. Database systems

An online retail company is trying to assess the performance of its DB systems and has asked you to investigate some of the operations. Consider a relation Seller (ID, Name, Country) – abbreviated as S – where the primary key (ID) is a 32-bit integer, the Name attribute is a 54-byte (fixed length) string, and Country is a 16-bit integer. Further consider a relation Product(ID, ProductID, ManufacturerID, Price) – abbreviated as P – with ID being a foreign key to Seller's ID, ProductID and ManufacturerID being 64-bit integers, Price being a 32-bit float, and the first three attributes making up the relation's (composite) primary key.

Assume that both relations are stored in files on disk organised in 512-byte blocks, with each block having a 10-byte header. Assume that S has $r_S = 1,000$ tuples and that P has $r_P = 100,000$ tuples. Last, assume that Product is stored organised in a sequential file sorted by its primary key and Student is stored organised in a heap file. Finally, note that the database system adopts fixed-length records – i.e., each file record corresponds to one tuple of the relation and vice versa.

(a) Compute the blocking factors and the number of blocks required to store these relations. Show your work.

[2]

Solution: (Bookwork for knowledge of the concepts, Problem solving otherwise)

Each record (tuple) of S has size: 4 + 54 + 2 = 60 bytes. Thus, the blocking factor will be:

$$bfr_S = \left\lfloor \frac{block\ size - header\ size}{record\ size} \right\rfloor = \left\lfloor \frac{512-10}{60} \right\rfloor = 8 \text{ records per block.}$$

The file will then contain:

$$n_S = \left\lceil \frac{num\ tuples}{blocking\ factor} \right\rceil = \left\lceil \frac{1000}{8} \right\rceil = 125 \text{ blocks.}$$

Similarly, each record of P has size: 4 + 8 + 8 + 2 = 24 bytes. Thus, the blocking factor will be:

$$bfr_P = \left\lfloor \frac{block\ size - header\ size}{record\ size} \right\rfloor = \left\lfloor \frac{512-10}{24} \right\rfloor = 20 \text{ records per block.}$$

The file will then contain:

$$n_P = \left\lceil \frac{num\ tuples}{blocking\ factor} \right\rceil = \left\lceil \frac{100{,}000}{20} \right\rceil = 5,000 \text{ blocks.}$$

(1 mark for the computation of the blocking factors, 1 mark for the number of file blocks)

(b) Consider the following query:

```
SELECT S.Name, P.ID, P.Price FROM Seller as S, Product as P
WHERE S.ID = P.ID AND S.ID >= 6,000 and S.ID <= 6,199;
```

Assume that the memory of the database system can accommodate $n_B = 22$ blocks for processing, that all seller IDs in the query range exist in the database, and that all sellers have the same number of products on average.

Explain the query processing algorithm, taking care to include the file organisation in your reasoning. Then estimate the total expected cost (in number of block accesses) of the query above (Disregard the cost associated with the writing of the result set), of the following two approaches:

(i) First, assume that S is scanned at the outer loop and show your work: [9]

Solution: (Bookwork for knowledge of the concepts, Problem solving and Synthesis otherwise)
Let $NDV(a,A)$ be the number of distinct values of attribute a in relation A. Then, $NDV(ID,Seller) = 200$ (as only values between 6,000 and 6,199 are considered) and $NDV(ID,Product) = 200$ (foreign key). As S is stored in a heap file, we need to scan all of its blocks during join processing($n_S = 100$ blocks). However, as ID is the relation's primary key, only 200 records of S will participate in join processing, hence for the purposes of this query $r'_S = NDV(ID,Seller) = 200$.

[**1 mark for realising that not all records of S participate in the join result, 1 mark for n_S, 1 mark for r'_S**]

On the other hand, P is stored in a sequential file sorted by primary key. As ID is the first component of the sort (primary) key, all records of interest will be stored contiguously on disk. Hence, scanning P consists of a binary search to locate the block storing the first record with the lowest ID in the range of interest, followed by a sequential scan until we find the first block storing a record with ID outside the range of interest. For the first, binary search, part, we'd need to read $\lceil log_2(5000) \rceil = 13$ blocks. For the second, sequential scan, part, when P is the inner relation, we need to read $r_{P_{inner}} = r_P / r_S = 100,000/1,000 = 100$ records on average per value of ID, translating to 5 blocks ($bfr_P = 20$) per value of ID, or $n_{P_{inner}} = 18$ blocks.

[**1 mark for $n_{P_{inner}}$ and 1 marks for $r_{P_{inner}}$**]

Note 1: *Students may opt to name the above differently and compute them as part of the discussion of the two approaches below. Full marks for these components should be awarded as long as the students have realised that not all records of S participate in the join processing, that P shouldn't be fully scanned but a binary* *search should be used instead, and the relevant quantities have been computed using correct formulae.*

If the outer loop in the nested-loop join scans over relation S, then we need to:

1. Scan all blocks of S, $n_B - 2$ blocks at a time (the latter not being significant in this case).
2. For each ID value in each such block of S, read all blocks of P with that ID value (one at a time) and produce join results.
3. Store join results in the output block until full; flush and continue.

Then, the cost is:
$n'_S + r'_S \times n'_{P_{inner}} = 125 + 200 * 18 = 3725$ block accesses.

[**3 marks for algorithm – one per step; 1 mark for computation**]

Note 2: *Students may opt to blindly apply the nested loop join equations. In that case, the cost for S being the outer relation would be:*
$n_S + \left\lceil \frac{n_S}{n_B - 2} \right\rceil \times n_P = 125 + 7 * 5000 = 20100$ block accesses. *This is an obviously suboptimal solution and should thus receive only partial marks, as discussed above.*

(ii) Second, assume instead that P is scanned at the outer loop and show your work: [9]

Solution: If P is the outer relation, then we'd need again 13 block accesses to get to the first block containing a record with ID=6,000, plus a sequential scan till we find the first block with a record with ID=6,200; that is, $n_{P_{outer}} = NDV(ID, Product) \times 5 = 200 \times 5 = 1000$ blocks, mapping to $r_{P_{outer}} = 25,000$ records ($r_{P_{outer}} = blocks * bfr_P = 1000 * 25$), and a grand total of $n'_{P_{outer}} = 1013$ blocks.

[**2 marks for realising** P **shouldn't be fully scanned, 1 mark for** $n_{P_{outer}}$**, 1 marks for** $n'_{P_{outer}}$**, 1 marks for** $r_{P_{outer}}$]

Since the outer loop in the nested-loop join scans over relation P, then we need to:

1. Scan all blocks of P, $n_B - 2$ blocks at a time.
2. For each such block, scan all blocks of S for matches and produce join results.
3. Store join results in the output block until full; flush and continue.

Then, the cost is:
$n'_{P_{outer}} + \left\lceil \frac{n'_{P_{outer}}}{n_B - 2} \right\rceil \times n'_S = 1013 + 50 \times 125 = 7263$ block accesses.

[**3 marks for algorithm – one per step; 1 mark for computation**]

Note: Again, Students may opt to blindly apply the nested loop join equations. In that case, the cost for $S = P$ being the outer relation would be:
$n_P + \left\lceil \frac{n_P}{n_B - 2} \right\rceil \times n_S = 5000 + 250 * 125 = 36250$ block accesses. This is an obviously suboptimal solution and should thus receive only partial marks, as discussed above.

Mark table

Question	Points	Score
1	20	
2	20	
3	20	
Total:	60	

END OF QUESTION PAPER